Upskilling Week
Barcelona

Mobile WorldCapital Barcelona

15:00h - 20:00h
Bravo Robots

# RL en MuJoCo: desde el entrenamiento hasta el despliegue de políticas para el MicroDuck

Impulsat per

Mobile WorldCapital Barcelona

Amb la col·laboració de

Barcelona | Barcelona Activa | Generalitat de Catalunya

Powered by

TALENT ARENA

# RL en MuJoCo: del entrenamiento al despliegue de políticas

Un pequeño robot bípedo, el MicroDuck, aprende a jugar a la rayuela.



Simulación en MuJoCo · Windows sin GPU · Bravo Robots

# Un robot que juega a la rayuela, y tú sabrás cómo

**1. Simulación** (bloque 1). Desenvolverte en MuJoCo y crear entornos propios.

**2. Fundamentos de RL** (bloque 2). Agente, entorno, observaciones, acciones, recompensas y PPO.

**3. Práctica** (bloque 3). Diseñar la recompensa, entrenar y ver cómo cada cambio altera la conducta.

**4. ROS 2** (bloque 4). Nodos, topics y lanzadores para desplegar políticas.

**5. Todo junto** (bloque 4). Una partida completa en simulación, con visión, control y teleoperación.

| Hora | Bloque | Min |
|---|---|---|
| 15:00 | Arranque e ignición | 20 |
| 15:20 | 1 · Modelar: el robot y su mundo | 65 |
| 16:25 | Descanso | 10 |
| 16:35 | 2 · El entorno y el RL | 40 |
| 17:15 | 3 · Entrenar: la voltereta | 55 |
| 18:10 | Descanso | 10 |
| 18:20 | 3 · Entrenar: configúralo | 35 |
| 18:55 | 4 · Desplegar: ROS 2 y la partida | 35 |
| 19:30 | 4 · Desplegar: sim-to-real y cierre | 30 |

El camino de hoy: modelar → entorno → entrenar → desplegar… y volver atrás: casi nunca sale a la primera.

LO PRIMERO DE TODO · 10 MIN

# Prepara tu PC ahora

**1.** Crea una carpeta vacía **C:\taller** y ábrela con **VS Code**.

**2.** Abre la terminal: **Ctrl+ñ** (Ctrl+` en teclado inglés).

**3.** Clona dentro (con el punto final):

```
git clone --depth 1 https://github.com/david2526IA/taller_microduck_windows .
```

**4.** Instala, sin tocar el teclado (2-3 min):

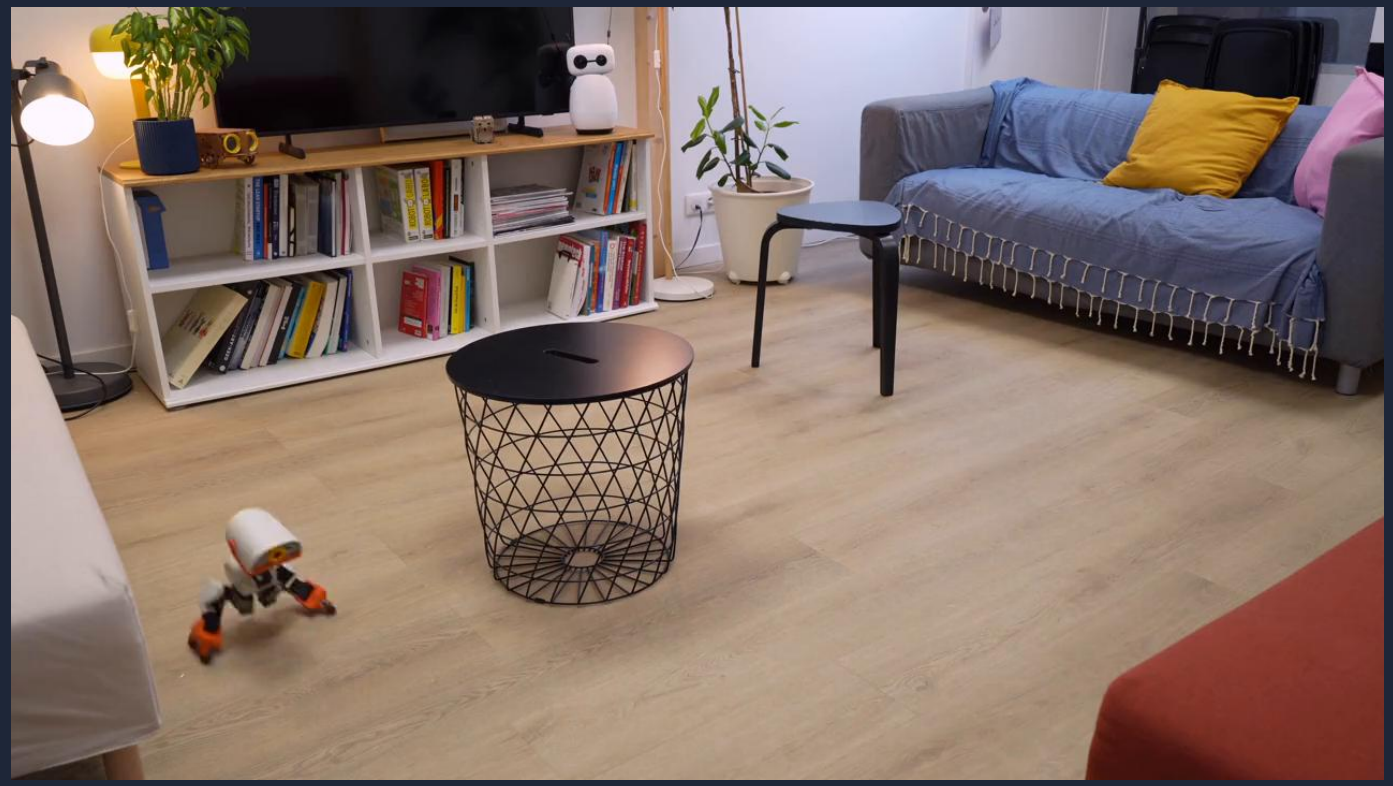
```
.\INSTALAR.bat
```

**5.** Debe acabar en **TODO LISTO**. Luego abre **PROGRAMA.md**.

**Escanéalo para abrir el repositorio**

Sin git: en GitHub, Code > Download ZIP, y descomprime en C:\taller. Nunca dentro de OneDrive.

# ¿Cómo ha aprendido a patinar?



Robot Microduck real · vídeo: Pollen Robotics

## ¿Cómo lo ha aprendido?

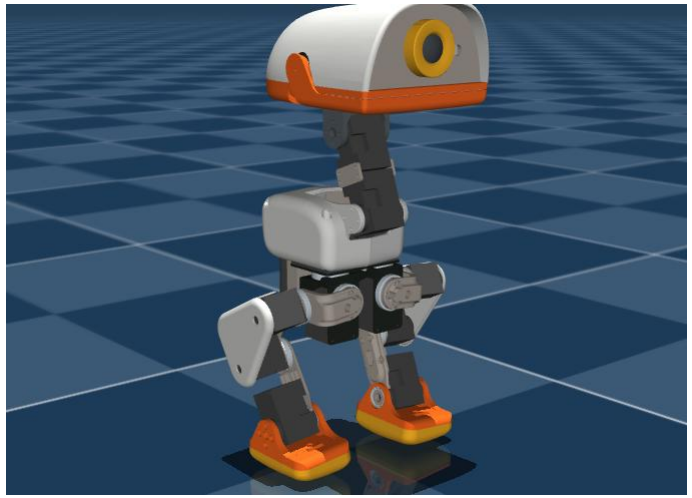
**1. En simulación.** Un robot virtual con ruedas bajo los pies.

**2. Por ensayo y error.** Miles de intentos a la vez en una GPU.

**3. Con una recompensa.** Puntos por seguir la velocidad pedida sin caerse.

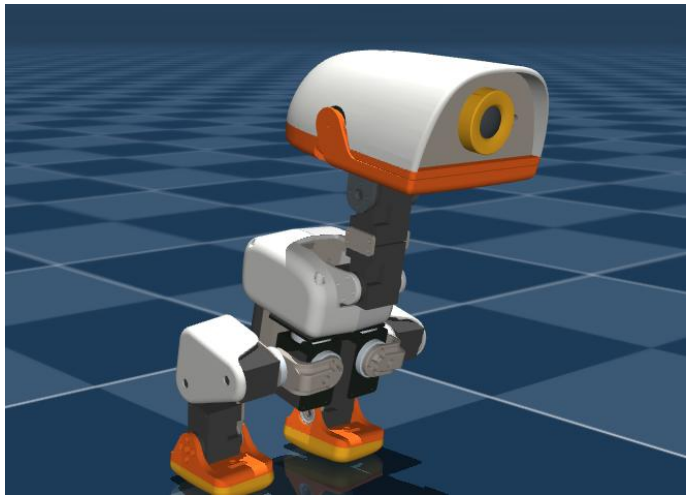
**4. Y al robot real.** El cerebro aprendido se copia tal cual.

# Sin cerebro, se cae en menos de un segundo



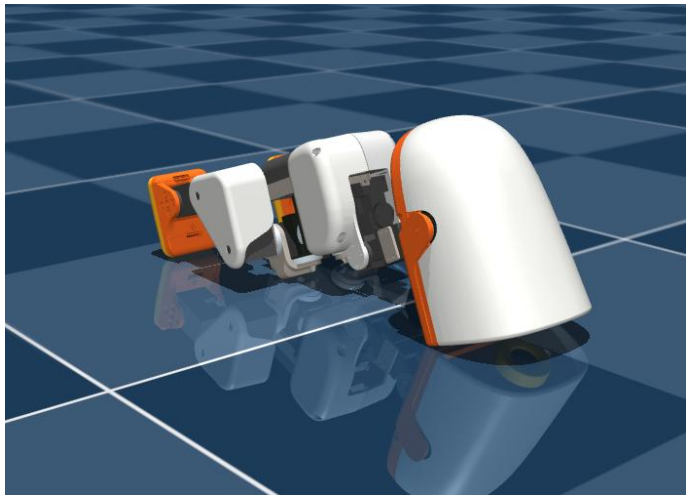
0 s

## De pie



0,4 s

## Se inclina



1 s

## En el suelo

Aunque le pidamos «mantén la postura», sin IA no aguanta.

# Un laboratorio de IA física en tu PC

1

## Python

El lenguaje de todo el taller. Exactamente la 3.12.

2

## Una caja aislada

Un entorno virtual con sus propias librerías.

3

## El simulador

MuJoCo calcula la física del robot.

4

## El visor

La ventana 3D donde lo ves moverse.

No hace falta una GPU: todo corre en la CPU de un PC normal.

# El mapa de tu laboratorio

| Carpeta o fichero | Qué es | ¿Lo tocas? |
|---|---|---|
| PROGRAMA.md | El orden del taller: qué archivo toca en cada momento | Se lee |
| INSTALAR.bat · COMPROBAR.bat | Preparar el PC y comprobar que todo funciona | Una vez |
| exercises_rayuela/ | Los ejercicios del taller, con parámetros arriba | Sí: las constantes |
| workshop-ia-fisica/ | El robot, los cerebros (.onnx), ejercicios n1-n3 y entrenar/ | Solo scene.xml |
| rayuela-oficial/ | La Rayuela oficial de Bravo Robots (simulación, visión, control) | No: se lee |
| windows/ | Lo que hace funcionar todo en Windows: lanzador y ROS 2 en miniatura | No |
| PROBAR.md · README.md | Cada ejercicio y sus parámetros | Se lee |

Cuerpo, mundo, cerebro y ejercicios: cuatro cosas que se cambian por separado.

# Así se trabaja en cada ejercicio

1

## Abre

El ejercicio en VS Code. Arriba hay unas constantes en mayúsculas.

2

## Cambia UN valor

Solo uno, para saber qué lo causó. Antes, predice qué pasará.

3

## Guarda y cierra

Ctrl+S, y cierra la ventana del robot: no se recarga sola.

4

## Lanza y observa

Ejecútalo otra vez. ¿Acertaste?

Si no cambia nada: ¿guardaste? ¿cerraste la ventana?

PRÁCTICA · 15 MIN

# El robot y el pato sin cerebro

**Antes de cambiar nada, predice** y apúntalo en **CUADERNO.md**: así se aprende.

**n0_00_pato_sin_cerebro.py** (esencial): ¿en cuánto tiempo se cae?

**Si sobra tiempo:** n1_00 (POSE: STAND, SIT, FOLD; empújalo con Ctrl + clic derecho) y n1_01 (PIEZA).

Cambia **FRICCION**, **FACTOR_MASA**, **EMPUJON** o **VELOCIDAD** y repite. ¿Algo lo salva?



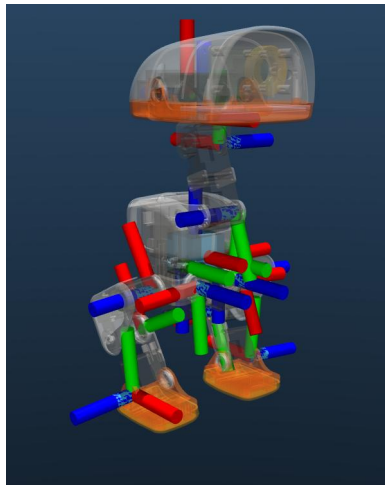
# Cinco etapas para crear un gemelo digital



1

## Modelo 3D

Las piezas.



2

## URDF

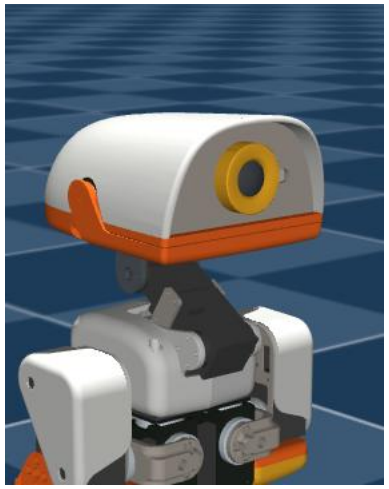
El esqueleto.



3

## MJCF

La física.



4

## Escena

El mundo.



5

## Validación

Se sostiene.

# El Microduck es un árbol de piezas

**Tronco**

Base libre: se mueve por el mundo

**PIERNA IZQUIERDA**

**Cadera** — 3 motores → **Rodilla** — 1 motor → **Tobillo** — 1 motor

**5** motores

**PIERNA DERECHA**

**Cadera** — 3 motores → **Rodilla** — 1 motor → **Tobillo** — 1 motor

**5** motores

**CUELLO Y CABEZA**

**Cuello** — 1 motor → **Cabeza** — 3 motores

**4** motores

**14** motores en total

14 acciones de la política

# MJCF: añadir la física

El esqueleto no basta: el simulador necesita saber cuánto pesa cada pieza y cómo empuja cada motor.

**Masas e inercias:** las cajas rojas de la imagen.

**Colisiones:** qué piezas chocan con qué.

**Actuadores:** los motores, en negro.

**URDF describe cómo está montado. MJCF añade cómo se comporta.**



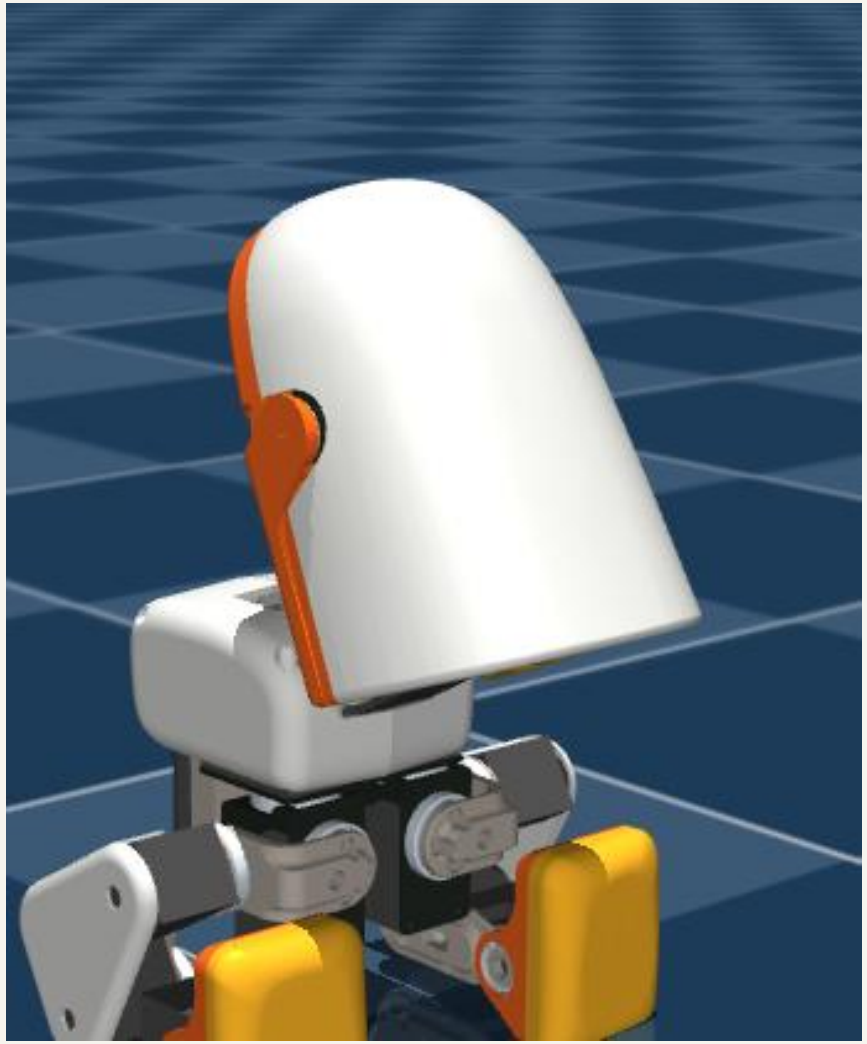
# MuJoCo: el simulador de física

MuJoCo (Multi-Joint dynamics with Contact) es un simulador creado para robótica. Hoy es de código abierto.

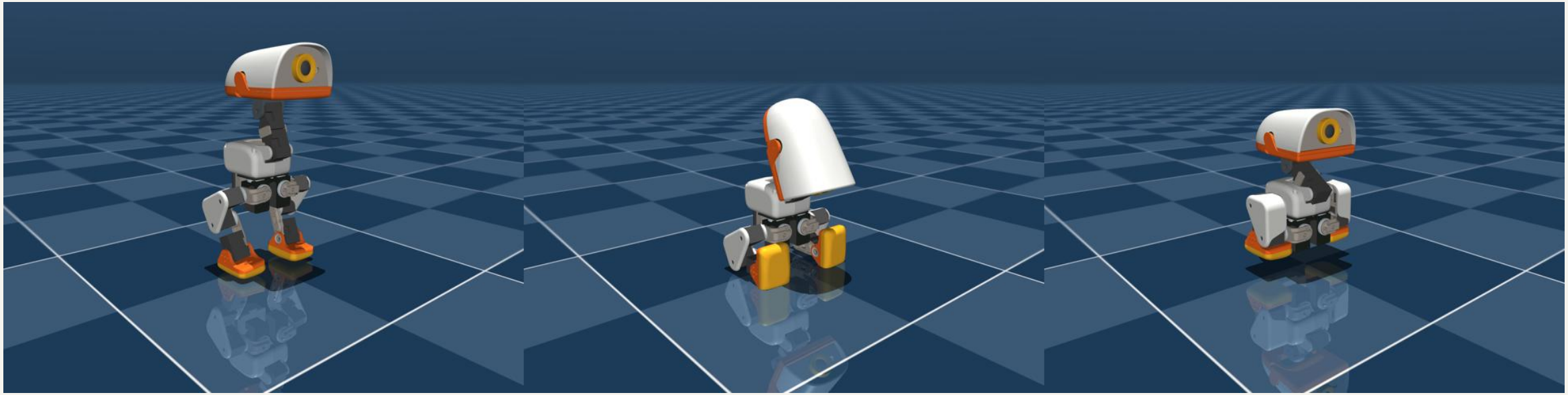
**Qué calcula:** cómo se mueve cada pieza y cómo choca con el suelo y con las demás.

**Cómo lo hace:** muchísimos pasos por segundo, cada uno de una fracción de milisegundo.

**En GPU:** MuJoCo Warp, de NVIDIA, simula miles de robots a la vez.



# La escena: el mundo del robot



**Suelo, luz y cámara.** Todo lo que rodea al robot.

**Posturas guardadas.** De pie, sentado y plegado.

**Objetos.** Una versión con pelota para patear.

# Crea tu propio mundo

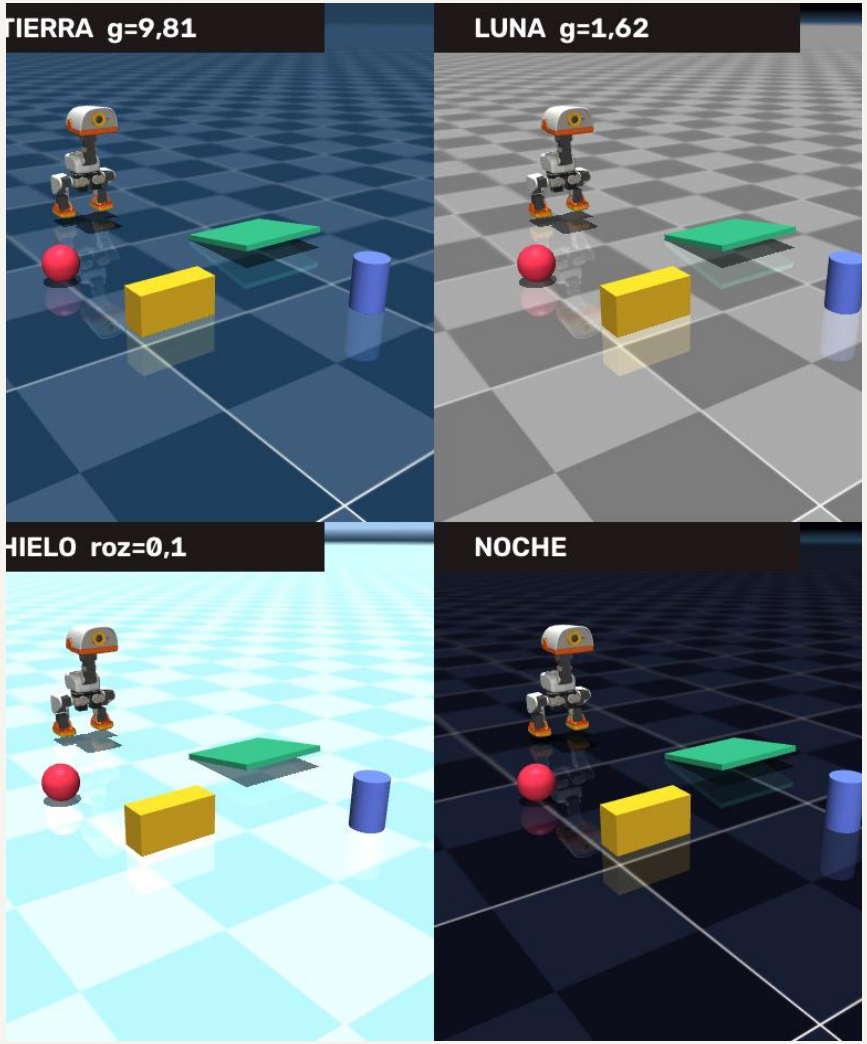
Un mundo MuJoCo se **describe** con piezas: suelo, cielo, luz, gravedad, objetos y un robot.

Abre **n1_02_tu_escena.py**. Prueba **MUNDO** = "luna", "hielo", "marte", "noche"...

Cambia a mano **GRAVEDAD**, **ROZAMIENTO_SUELO** y los colores del suelo y del cielo.

Añade **OBJETOS**: caja, rampa, esfera o cilindro. Reto: un laberinto hasta la casilla 9.

El cerebro se entrenó en la Tierra: ¿qué nota en otro mundo?

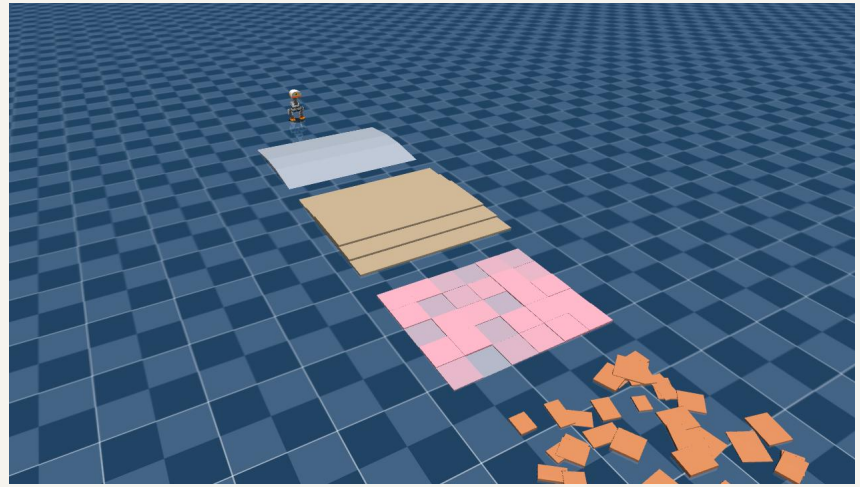


# Del GTC de Berlín a tu PC: un circuito en crescendo

| Nivel del circuito | alpha_walking | rough_walk_e | rough_walk_g |
|---|---|---|---|
| 0 · llano | 6/6 | 6/6 | 6/6 |
| 1 · + rampa | 6/6 | 6/6 | 6/6 |
| 2 · + escalones | 6/6 | 6/6 | 3/6 |
| 3 · + cuadrícula irregular | 6/6 | 6/6 | 3/6 |
| 4 · + escombros | 3/6 | 5/6 | 3/6 |

Recorridos completados de 6, en MuJoCo y en CPU. **Pocas pruebas: mucho ruido.**

**n4_05** lo ves andar y cambias el NIVEL. **n4_06** lo valida con números.



**Replicamos:** el circuito, las políticas ya entrenadas y la validación.

**No replicamos:** el agente que lo monta, la cámara de profundidad, Isaac Sim ni el entrenamiento en GPU.

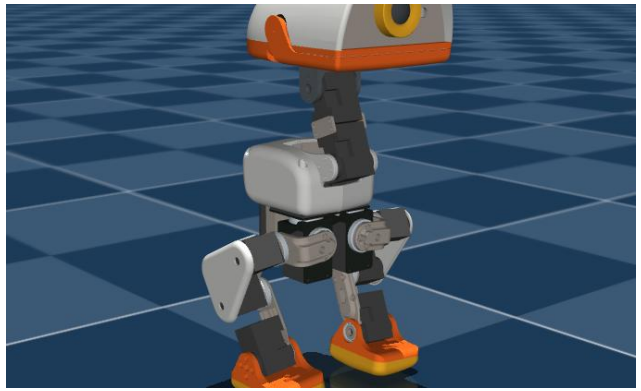
# Aprender es como montar en bici

Nadie te da ecuaciones de equilibrio. Te subes, te caes, ajustas y lo intentas otra vez.

**Con el robot pasa exactamente igual.**



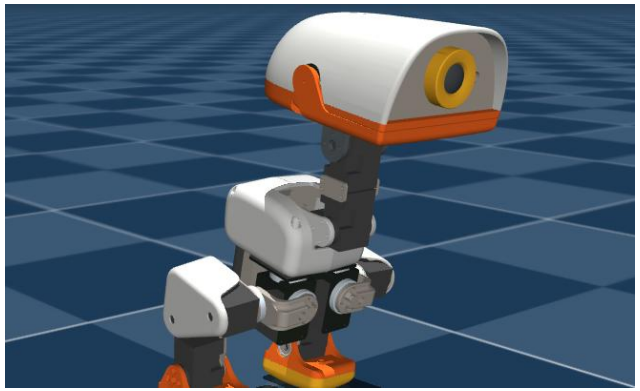
# El bucle de aprender



1

## Intentar

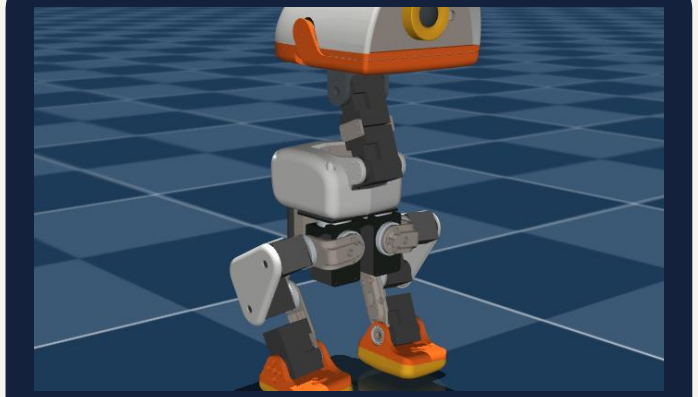
Mueve los motores.



2

## Ver el error

Se inclina y cae.



3

## Ajustar

Corrige un poco y repite.

Toda la IA de este taller es este bucle, repetido millones de veces.

# Una red neuronal es una función con botones

### Entran 61 números

Lo que siente el robot: inclinación, giro del cuerpo, posición de los motores y la orden de velocidad.

### Miles de botones

Son los pesos de la red. Aprender es ajustarlos.

### Salen 14 números

Los ángulos que debe tomar cada motor.

Cada 20 milisegundos: entran los sensores y salen los motores.

# Dentro del cerebro: 197.896 números

**Sensores**
giro, inclinación, motores

→

**Observación**
61 números

→

**Cerebro**
red neuronal

→

**Acciones**
14 órdenes

→

**Motores**
XL330 con control propio

→

**Física**
MuJoCo, 5 ms por paso

↺ se repite 50 veces por segundo: cada decisión del cerebro = 4 pasos de física de 5 ms

## La red

61 entradas → normalizar → 512 → 256 → 128 → 14 órdenes

Activación ELU. Son sumas y multiplicaciones: 197.896 pesos.

## Las 61 entradas

3 giro · 3 inclinación · 14 posición

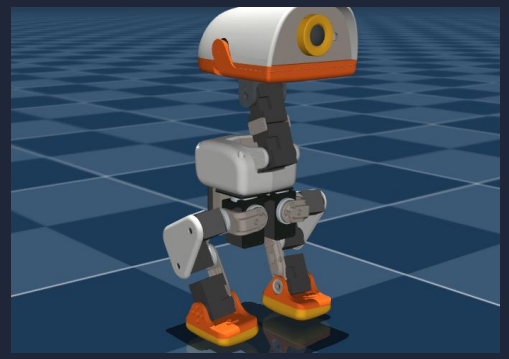
14 velocidad · 14 última orden

13 de orden (movimiento, cabeza, cuerpo)

## Lo que lleva el fichero

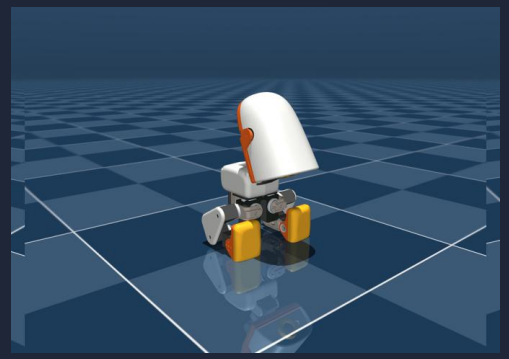
Nombres de las 14 articulaciones, pose por defecto y los nombres de cada entrada: el .onnx se describe a sí mismo.

# Cuatro piezas para aprender sin profesor



## Agente

Quien decide: la política del robot.



## Entorno

El mundo donde actúa: el simulador MuJoCo.

ACCIÓN

## 14 ángulos

Uno por motor, 50 veces por segundo.

RECOMPENSA

## Un número

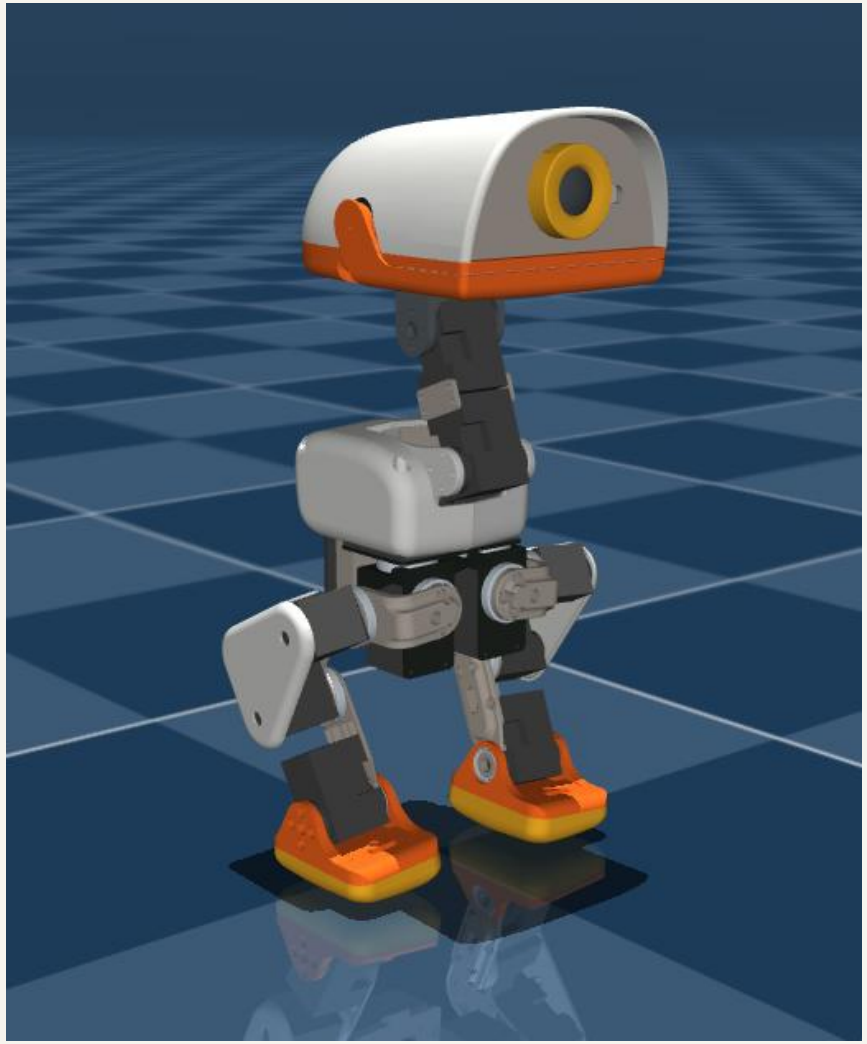
¿Lo hizo bien? Más alto, mejor.

# ¿Qué le damos como recompensa?

**Premio por seguir la orden.** Si le pedimos avanzar a cierta velocidad, gana puntos cuanto más se acerque.

**Castigo por caerse.** Si se cae, el intento termina y pierde lo que podía ganar.

**Nadie dice cómo andar. Solo qué cuenta como hacerlo bien.**



# PPO: aprender con pasos pequeños

PPO (Proximal Policy Optimization) es el algoritmo con el que se entrena el Microduck. Es de los más usados en robótica.

1

## Jugar

Miles de robots prueban la política actual.

2

## Puntuar

Se ve qué acciones salieron mejor de lo esperado.

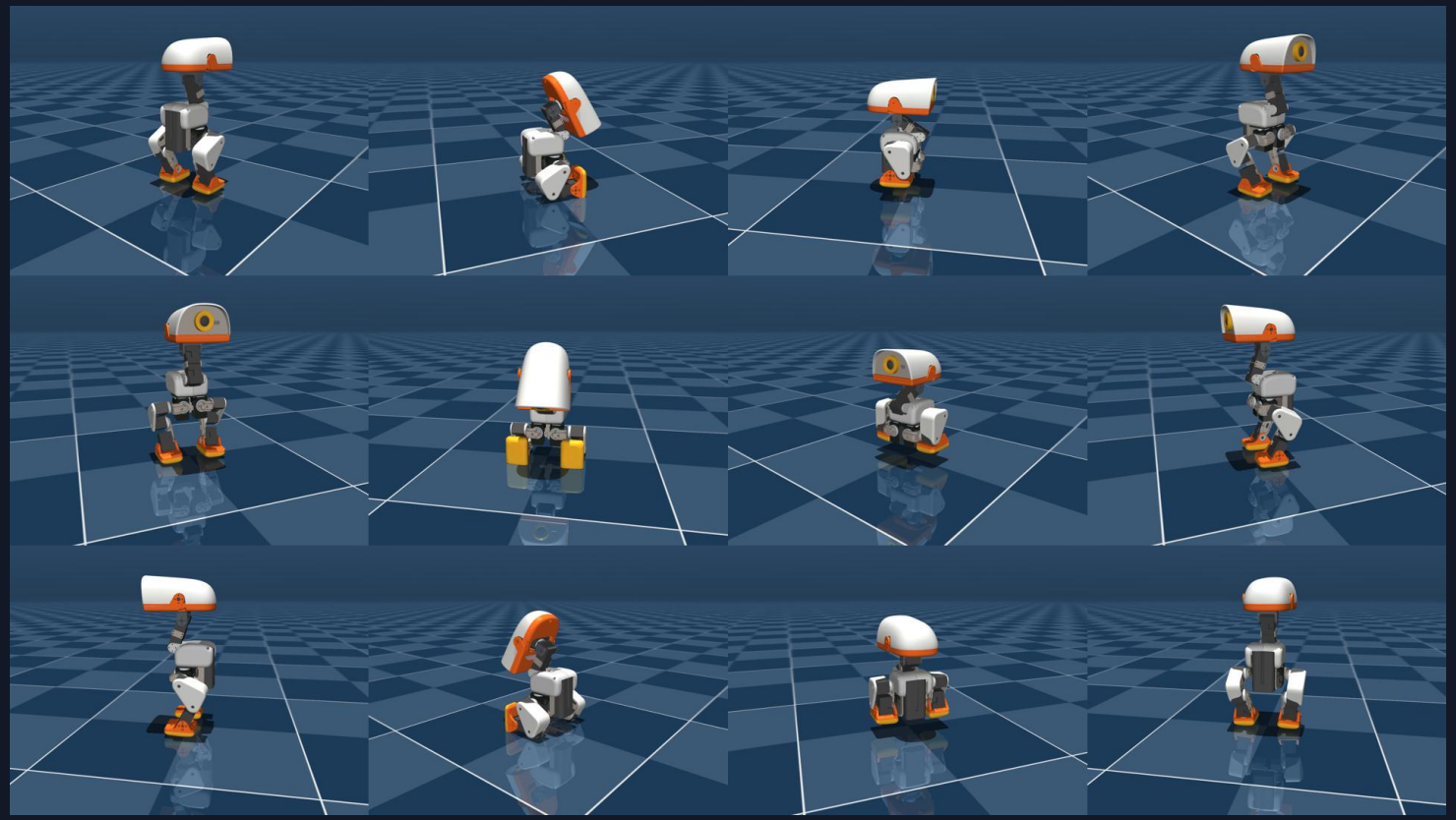
3

## Mejorar un poco

Se refuerzan esas acciones, con un límite de cambio por paso.

Ese límite (en el Microduck, un 20 %) evita que un mal paso estropee lo ya aprendido.

# Miles de patos aprenden a la vez



**4096**

robots simulados a la vez en una sola GPU NVIDIA.

Una hora o dos bastan para una marcha usable.

Cada recuadro de la imagen es un robot distinto.

# Toca el cerebro

n3_00

## Con cerebro

Anda, se sienta, rueda y se recupera de empujones.

Empújalo con Ctrl + clic derecho.

n2_03 · n2_00

## Qué siente y qué ordena

61 entradas y 14 salidas en vivo.

QUITAR_SENTIDO = "inclinacion": ¿se cae?

n2_01

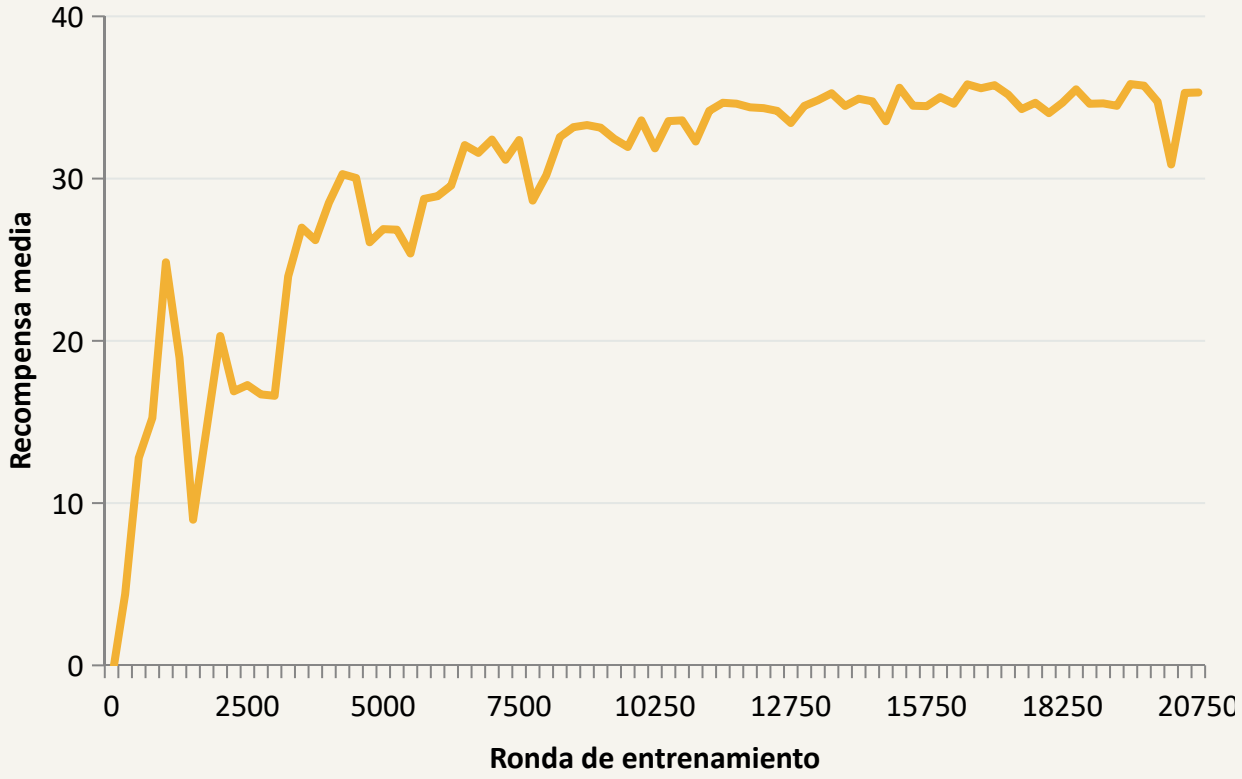
## Cómo se le puntúa

Piezas × pesos = nota.

PESO_AVANZAR = 5 y el resto 0: ¿qué truco aprende?

Cambia UN valor, guarda, cierra la ventana del robot y lanza otra vez.

# Una voltereta, nueve checkpoints




| Checkpoint | Salen bien | Qué se ve |
|---|---|---|
| 0 | 0 de 10 | Se queda quieto |
| 500 | 0 de 10 | Todavía nada |
| 750 | 10 de 10 | Aparece la voltereta |
| 1.250 | 9 de 10 | Casi siempre |
| 3.000 | 4 de 10 | Irregular |
| 7.000 | 5 de 10 | Sigue irregular |
| 11.000 | 10 de 10 | Estable |
| 20.000 | 3 de 10 | Peor que el 11.000 |
| 20.750 | 10 de 10 | El final |

n3_07: cambia CHECKPOINT y pulsa R · n3_08: mídelos con números. Entrenar más no siempre mejora.

# El ciclo del entrenamiento

1

## Tarea

Qué debe aprender y qué se premia.

2

## Jugar

4096 robots prueban el cerebro actual.

3

## Puntuar

Cada intento recibe una recompensa.

4

## Ajustar

PPO mueve los botones hacia lo que dio más puntos.

5

## Repetir

Cientos de rondas. Se guarda una copia cada 25.

Toda la IA de este taller es este bucle, repetido millones de veces.

# Los botones del entrenamiento

| Botón | Valor | Qué controla |
|---|---|---|
| num-envs | 4096 | Robots a la vez. Más robots, más experiencia por ronda. |
| learning-rate | 0,001 | Tamaño de cada ajuste. Muy alto se descontrola; muy bajo casi no avanza. |
| entropy-coef | 0,01 | Premio por probar cosas distintas (exploración). |
| gamma | 0,99 | Cuánto importa el futuro frente a lo inmediato. |
| clip | 0,2 | Límite de cambio en cada ajuste: protege lo ya aprendido. |
| max-iterations | 300 | Cuántas rondas de entrenamiento. |

Cada botón es una opción del comando de entrenamiento.

# La configuración de un entrenamiento

```
# tarea
Mjlab-StandUp-Flat-MicroDuck
# entorno  (env.yaml)
num_envs: 4096           # robots a la vez
decimation: 4 × 0,005 s  # = 50 Hz
episode_length_s: 6
# algoritmo PPO  (agent.yaml)
learning_rate: 0.001  (adaptive)
gamma: 0.99    lam: 0.95
clip_param: 0.2    entropy_coef: 0.01
num_learning_epochs: 5    num_mini_batches: 4
num_steps_per_env: 24
# red  (actor y critic)
hidden_dims: [512, 256, 128]   activation: elu
```

## Dos ficheros, todo el entrenamiento

Cada carrera guarda su configuración en params/env.yaml y params/agent.yaml.

Cualquier valor se cambia al lanzar:

--agent.algorithm.learning-rate 0.0005

--env.scene.num-envs 2048

--agent.max-iterations 500

# La recompensa: una suma con pesos

| Pieza de la recompensa | Peso | Qué premia o castiga |
|---|---|---|
| height_stand_sharp | +1,0 | Tener el tronco a la altura de pie |
| upright_sharp | +1,5 | Estar bien vertical (margen de unos 6°) |
| standing_composite | +3,75 | Puntuación combinada de estar de pie |
| pose_stand_legs | +2,0 | Piernas en la postura de pie |
| action_rate_l2 | −0,1 | Castiga sacudir los motores |
| dof_pos_limits | −1,0 | Castiga forzar las articulaciones al límite |
| self_collisions | −1,0 | Castiga chocar consigo mismo |

Cambiar un peso cambia lo que aprende: --env.rewards.upright-sharp.weight 0

# La recompensa de la patada

```
recompensa =
  − PESO_PRECISION × distancia a la casilla     (2.0)
  + BONUS_ACIERTO  × cae dentro de la casilla   (1.0)
  − PENALIZA_CAIDA × se cae el pato             (3.0)
  − PESO_FUERZA    × velocidad²                 (0.0)
```

## Un episodio = una patada

**Observación:** dónde está la casilla objetivo.

**Acción:** con qué pie y a qué velocidad (m/s) sale la pelota.

**Recompensa:** la suma de arriba.

El cerebro de patada (.onnx) ya viene hecho: aquí se entrena QUÉ velocidad elegir.

Cambia un peso, vuelve a entrenar y mira qué cambia: es el diseño de la recompensa.

# PPO de verdad, en tu PC: el bucle completo

1 · Agente

## La política

Una red pequeña elige pie y velocidad según la casilla.

2 · Entorno

## MuJoCo

Simula la patada con el cerebro ONNX oficial.

3 · Recompensa

## La fórmula

Puntúa dónde cayó la pelota.

4 · PPO

## Ajusta

Repite lo que puntuó bien, sin cambiar demasiado de golpe.

**~1 s por patada simulada · 60 rondas × 12 patadas ≈ 3 minutos · sin GPU.**

Botones que vas a tocar: **LEARNING_RATE**, **CLIP**, **EXPLORACION**, **ITERACIONES**, **PATADAS_POR_RONDA** y los pesos de la recompensa.

# Si algo falla, ¿qué etapa tocas?

| Lo que ves | Etapa | Qué tocas | Lo que hemos medido |
|---|---|---|---|
| Se cae nada más empezar | Modelar | Postura, masa, fricción… o falta el cerebro | Sin cerebro, <1 s; ni la fricción ni la masa lo salvan |
| La curva no sube | Entrenar | Tasa de aprendizaje, robots, exploración | Tasa 0,01: se descontrola · 256 robots: 6,1 frente a 25,4 |
| Aprende con un truco raro | Entorno: recompensa | Los pesos, sobre todo los castigos | Voltereta sin castigo de velocidad: un latigazo |
| Solo acierta lo que vio | Entorno: qué ve | Más variedad al entrenar | Casillas 3 y 7: 2 de 9 frente a 8 |
| Va bien y luego empeora | Entrenar | Medir y elegir el checkpoint | Voltereta: 11.000 → 10/10 · 20.000 → 3/10 |
| Falla en otro mundo | Desplegar | Variedad, y que perciba lo que cambia | Un mundo: 4,2 · a ciegas: 2,0 · percibiendo: 6,8 |
| Desde cero cuesta demasiado | Entrenar | Partir de un cerebro que ya sabe | 800 rondas frente a 3.300 |

Medido en simulación con pocas semillas: orientativo. Lo importante es predecir, comprobar y volver atrás.

# Configura un entrenamiento real y llévatelo

```
uv run train Mjlab-Roulade-Flat-MicroDuck
   --env.scene.num-envs 2048
   --agent.max-iterations 2000
   --agent.save-interval 250
   --agent.algorithm.learning-rate 0.001
   --agent.algorithm.entropy-coef 0.01
   --agent.algorithm.gamma 0.99
MICRODUCK_REWARD_WEIGHTS="roulade_overspeed=0"
```

## Lo que haces en el PC

**n6_00** · La voltereta (el caso de partida)

**n6_01** · La Rayuela: patear a una casilla

**n6_02** · La cuerda: autojuego

**n6_03** · Los patines: otro cuerpo

**n6_04** · Obstáculos: terreno irregular

Cada uno lee la recompensa real, te deja cambiarla y escribe tu paquete.

Sin GPU se prepara todo; solo falta el botón. Siempre una prueba de humo antes del entrenamiento largo.

# No empieces de cero: parte de lo que ya sabe




La voltereta se entrenó de cero (20.750 rondas): es otra habilidad, no una variante del andar.

| Política | Caídas (110) | Circuito, nivel 4 |
|---|---|---|
| alpha_walking | 32 | 3 de 6 |
| rough_walk_e · 800 r. | 14 | 5 de 6 |
| rough_walk_g · 3.300 r. | 12 | 3 de 6 |

**Afinar 800 rondas** dio casi lo mismo que entrenar 3.300 desde cero, y **conserva el estilo de andar** (más seguro para el robot real).

En n6_05 eliges el caso y partes de la base o de cero: el comando cambia en dos líneas.

Menos rondas, menos GPU: partir de lo que ya sabe es lo habitual en RL con robots.

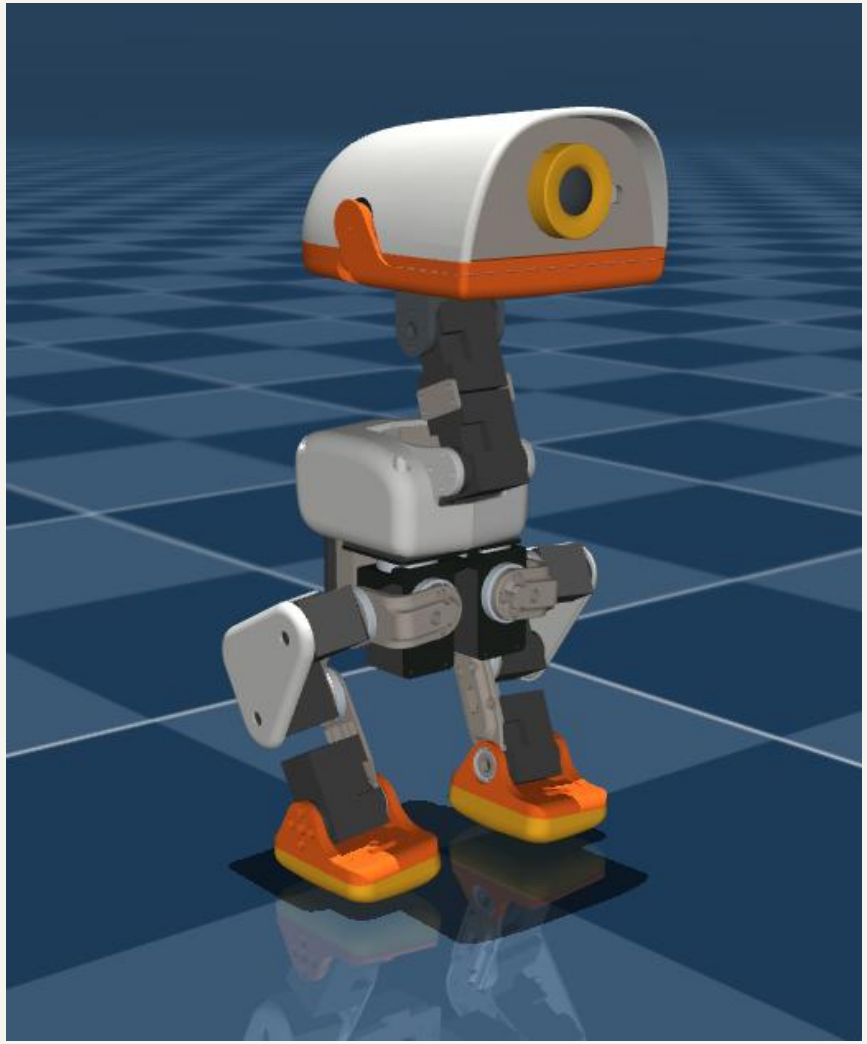
# Una política es un fichero

Después de entrenar, todo lo aprendido se guarda en un archivo: el cerebro del robot.

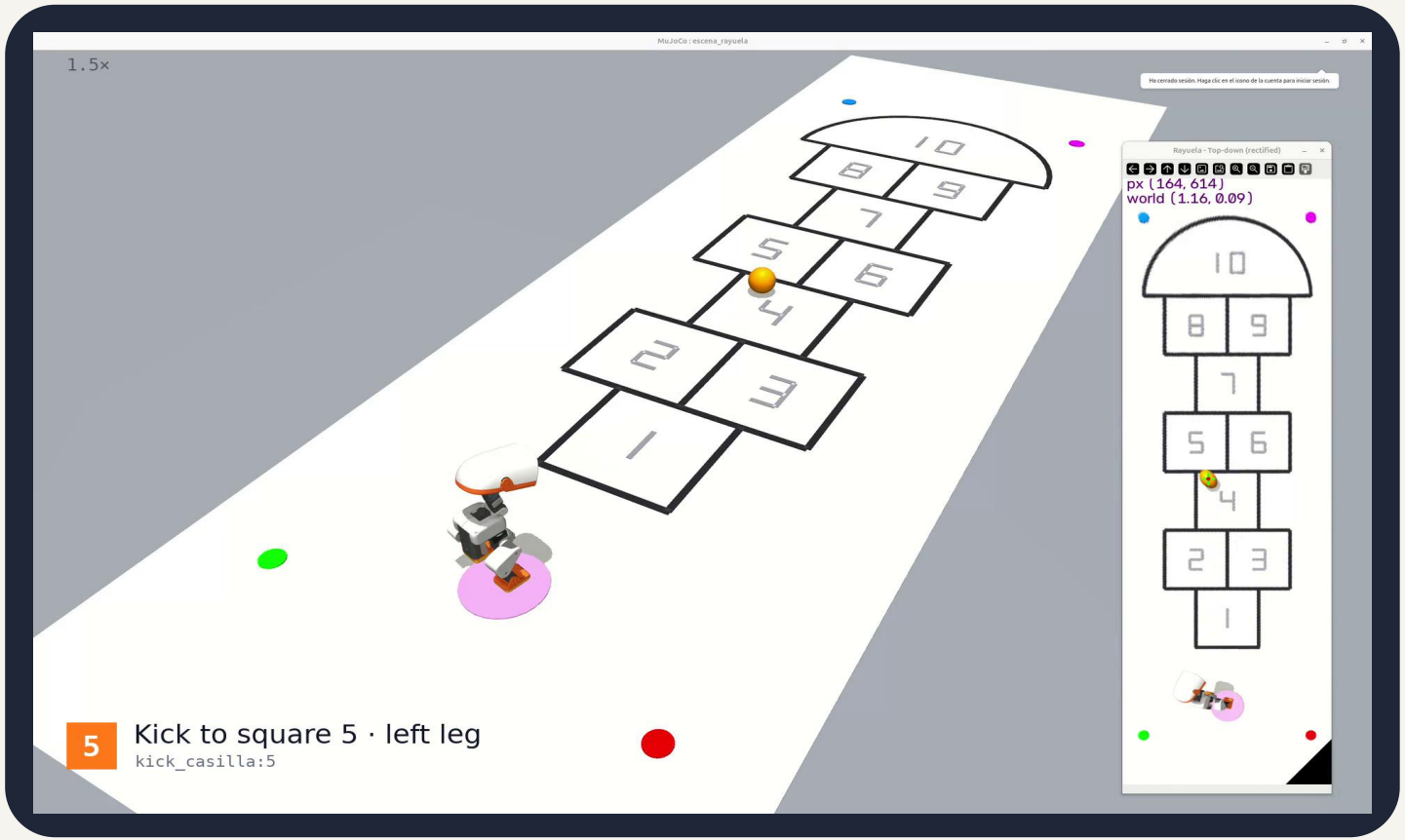
**Formato ONNX.** Un estándar para guardar redes entrenadas.

**Se ejecuta en cualquier PC.** No hace falta GPU para usarlo.

**Siempre 61 entradas y 14 salidas.** Por eso puedes cambiar de cerebro.



# Rayuela: el Microduck juega




Simulación en MuJoCo + ROS 2 · Bravo Robots

## Tres pasos, tres IA

**1. Patea.** Una política entrenada para golpear la pelota con la fuerza justa.

**2. Mira.** Una cámara calcula en qué casilla ha caído.

**3. Va.** Otras políticas lo llevan andando o rodando hasta allí.

**github.com/Bravo-Robots/microduck_rl-rayuela**

# ROS 2: nodos y topics

**Nodo**

## teleop

Publica:

/rayuela/cmd_vel

/rayuela/behavior_cmd

/rayuela/go_home

**Nodo**

## simulación

Publica:

/rayuela/duck_pose

/rayuela/current_policy

la imagen de la cámara

**Nodo**

## visión

Lee la imagen.

Publica:

/rayuela/target_casilla

**Nodo**

## control

Lee la casilla y la pose.

Publica la orden para llegar.

**Topic** = canal con nombre · **publicar** = poner un mensaje · **suscribirse** = recibirlo.

Nadie llama a nadie: cada nodo se puede apagar o cambiar sin tocar los demás.

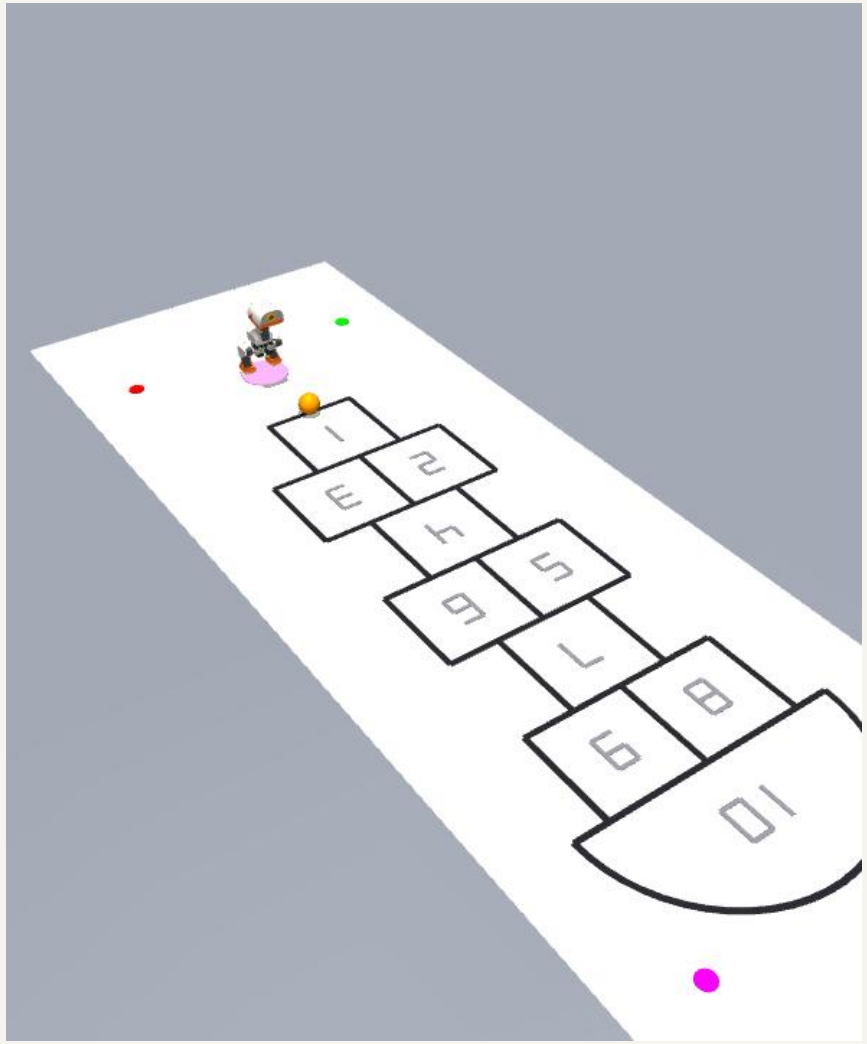
# Cómo encajan las políticas ONNX en la Rayuela

**Andar** y **quieto**: alpha_walking y alpha_standing.

**Patear:** ball_kick_speed (izquierdo y derecho): el cerebro recibe la velocidad de salida de la pelota.

**Voltereta, sentarse y recoger** del suelo: roulade, sitstand, ground_pick.

Todas: **61 entradas y 14 salidas**, un fichero .onnx. Un mismo programa las intercambia.



# Lanzadores: arrancar todo con una orden

```
# rayuela.launch.py (oficial, resumido)
Node(bridge_node)           # puente con la simulación
ExecuteProcess(sim_worker) # MuJoCo + cerebros
Node(vision_node,
     parameters={vis_rect, vis_raw})
Node(control_node)

# argumentos de lanzamiento
use_viewer  enable_vision  enable_control
```

## En Windows

**ROS 2 real** pide Ubuntu 22.04.

**windows/casos.py** lo imita en un solo proceso, con un rclpy en miniatura.

El código de **visión, control y teleop es el oficial**, sin cambiar una línea.

Es una imitación para entender nodos y topics; no hay red entre procesos ni ros2 topic.

# Juega la partida de la rayuela

n4_01 · ROS 2 en vivo

## Mira los topics

Tabla de mensajes por segundo.

VISION = False: ¿qué deja de moverse?

n4_00 · La partida

## Mete la pelota

Casilla objetivo, marcador y rondas.

k / l a tu velocidad; 1-9 automática.

Reto

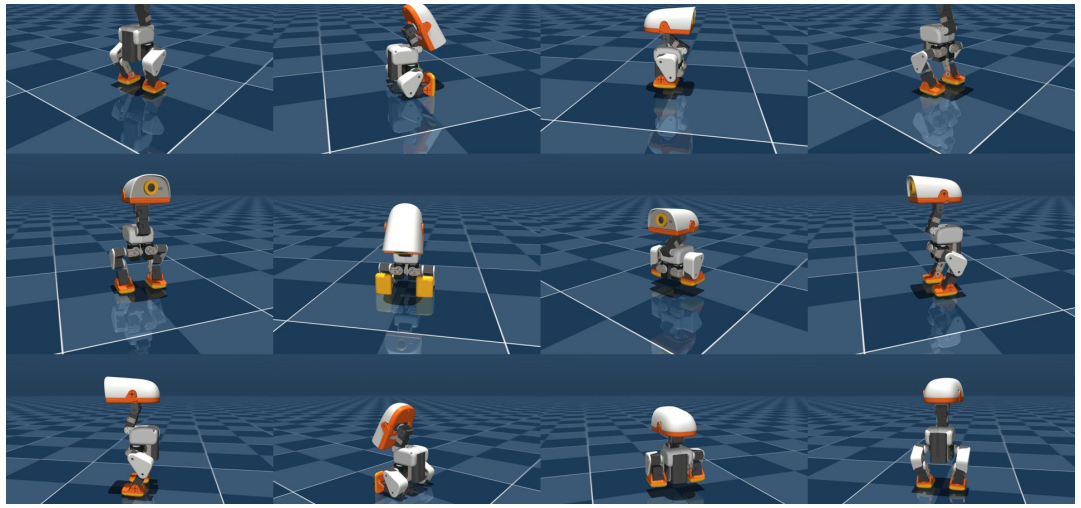
## Tu política contra la oficial

Pon TABLA_PROPIA = la que entrenaste.

¿Quién acierta más?

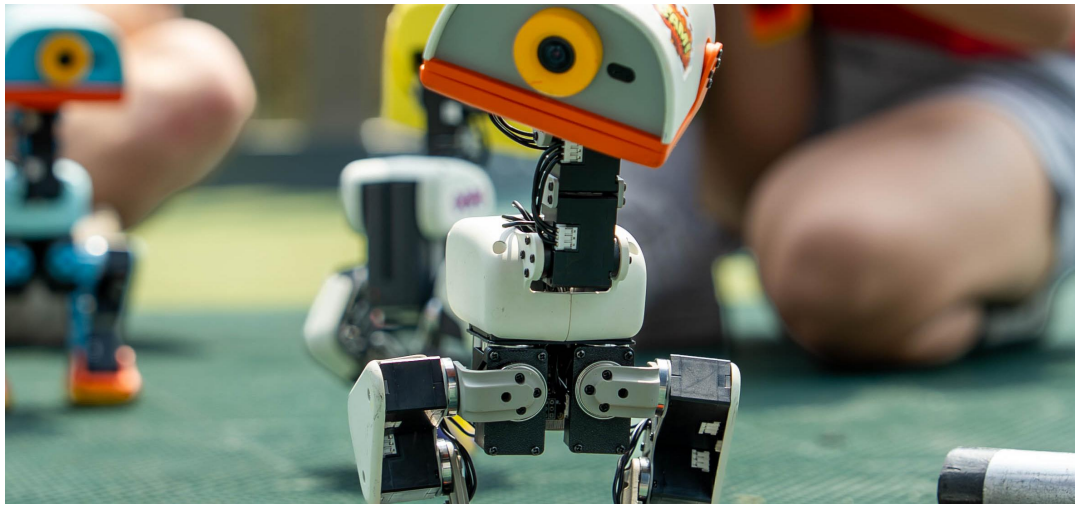
Teclas en la terminal: flechas, 1-9 patada a la casilla, h a casa, n ronda nueva, q salir.

# La brecha entre simulación y realidad



## Simulación

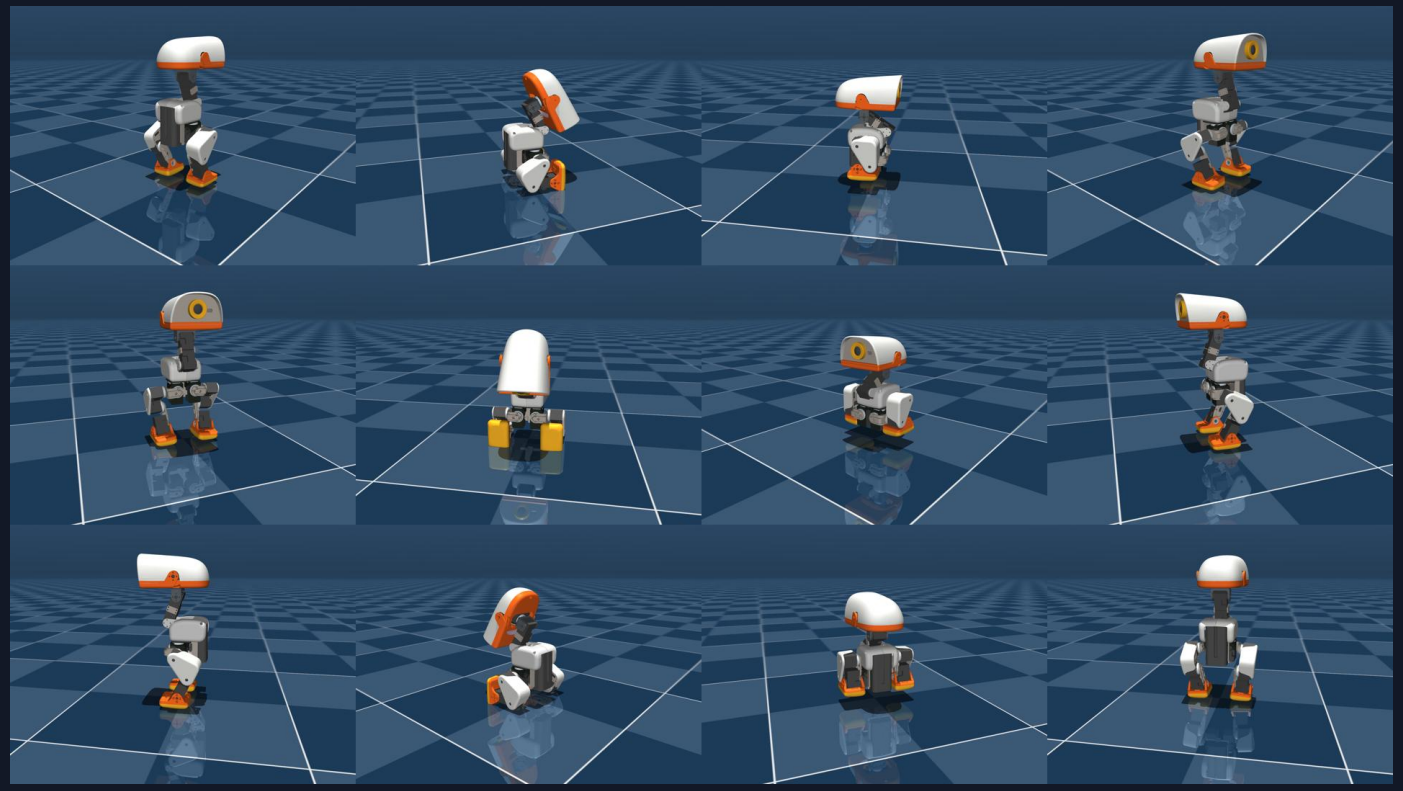
Física perfecta y repetible.



## Realidad

Holguras, batería, retrasos, suelos distintos.

# Entrenar con el mundo al azar



En cada intento se cambia al azar:

· masa y centro de masas

· fricción del suelo

· inercia de las articulaciones

· voltaje de la batería

· retraso de las órdenes

**Así no depende de un valor exacto.**

# Entrenada en un mundo, falla en otro

| Aciertos de 9 | Un solo mundo | Mundos al azar, a ciegas | Mundos al azar, percibiéndolos |
|---|---|---|---|
| Mundo normal | 8 | 3 | 8 |
| Rozamiento menor (-20 %) | 1 | 3 | 7 |
| Rozamiento mayor (+25 %) | 2 | 1 | 7 |
| Pelota más pesada (x1,4) | 5 | 2 | 7 |
| Gravedad menor (-20 %) | 7 | 2 | 8 |
| Todo a la vez | 2 | 1 | 4 |
| MEDIA | 4,2 | 2,0 | 6,8 |

Aleatorizar a ciegas no basta: la política tiene que percibir lo que cambia. Y entrenar con variedad cuesta más rondas.

# Tu lista antes de llevar una política al robot

| Nivel (cada uno añade una cosa) | alpha_walking | rough_walk_e | rough_walk_g |
|---|---|---|---|
| 0 · mundo ideal | 0/10 | 0/10 | 0/10 |
| 1 · + rozamiento al azar | 0/10 | 0/10 | 0/10 |
| 2 · + masa al azar | 0/10 | 0/10 | 0/10 |
| 3 · + batería baja | 0/10 | 0/10 | 0/10 |
| 4 · + retraso en los motores | 0/10 | 1/10 | 3/10 |
| 5 · + empujones | 4/10 | 3/10 | 4/10 |

Caídas en 10 recorridos de 10 s, en MuJoCo y en CPU. **Mucho ruido: pocas pruebas.**

## 1 · Contrato

61 entradas, 14 salidas, 32 bits, 64 MiB como máximo.

## 2 · Robustez

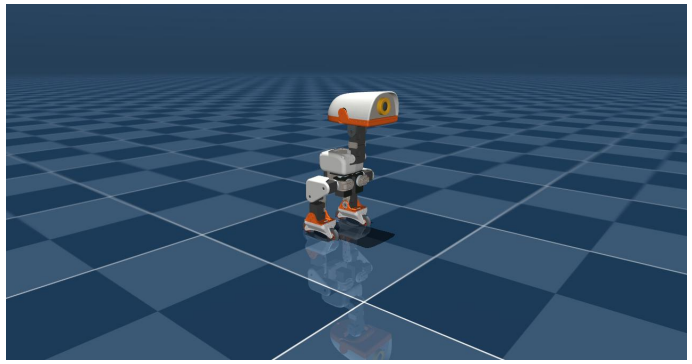
El mundo cada vez más sucio: ¿en qué nivel se cae?

## 3 · Despliegue

scp al robot, robotctl policy load walk, y vuelta atrás con policy reset.

Nada probado con el robot real: es la preparación.

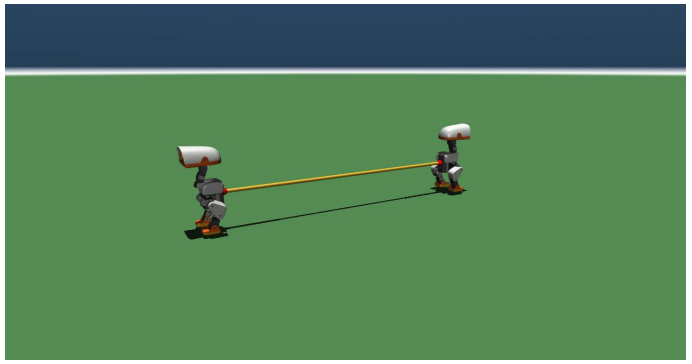
# Lo mismo, con otros cuerpos y otros mundos



n4_02 · Patines

## Otro cuerpo, otro cerebro

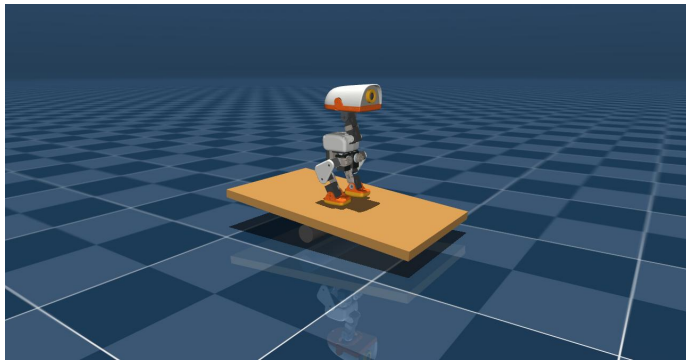
Batería, retraso, rozamiento.



n4_03 · La cuerda

## Un entorno de dos robots

Fuerza y masa de cada pato.



n4_04 · El equilibrio

## Tú lo empujas, él se corrige

Empujones, postura del cuerpo y balancín.

La cuerda es el duelo entrenado de patos_cuerda: se explica en «No empieces de cero» (bloque 3).

CIERRE

# Cinco ideas para llevarte

1

## Aprender

Error y ajuste, repetido muchas veces.

2

## Refuerzo

Probar y recibir una recompensa.

3

## Gemelo digital

Un cuerpo virtual donde equivocarse gratis.

4

## Política

El cerebro: un fichero que se puede copiar.

5

## Sim-to-real

El reto: que funcione fuera.

Entrenar el tuyo necesita una GPU NVIDIA, MuJoCo y el algoritmo PPO. Ya sabes qué hace cada uno.

PRÓXIMA MISIÓN

# Gracias

**Reentrena** el cerebro con una GPU NVIDIA.

**Cambia la recompensa** y mira qué aprende.

**Prueba otro robot** o MuJoCo Playground.



Recursos: pollen-robotics/microduck_rl · pollen-robotics/microduck-policies (Hugging Face) · tutorial de NVIDIA «Microduck con Isaac Lab» · MuJoCo y mjlab

Robot y mallas 3D: Pollen Robotics (CC BY-NC-SA 4.0). Fotos y vídeos reales: Pollen Robotics. Imágenes del simulador: renders propios.

Upskilling Week
Barcelona

# Thanks for coming!

## Please leave your feedback here.

It will take no more than 5 minutes

Impulsat per

Mobile WorldCapital Barcelona

Amb la col·laboració de

Barcelona | Barcelona Activa | Generalitat de Catalunya

Powered by

TALENT ARENA